MODULE 1 · LEVEL 300

AI in the SDLC & the GitHub Copilot Platform

Foundations for a practitioner audience - concepts, the agentic Copilot platform, and how it maps to this course

Day 1 · 90 minutes · Tool: GitHub Copilot (agentic) in VS Code, CLI, github.com ·

Where this module sits

approx 2 min

Module 1 builds the foundation and the shared tool. Each SDLC phase has its own dedicated module - we map, not deep-dive, today.

1	2	3	4	5	6	7
Intro + Copilot platform	**Requirements**	**Design**	**Development**	**Testing**	**Deployment**	**Maintenance**
TODAY	Day 1	Day 1	Day 2	Day 2	Day 3	Day 3

This 90 minutes: 3 parts - Foundations (~25 min), the GitHub Copilot platform (~40 min), and how it maps across the SDLC (~25 min).

Foundations

Concepts done right: GenAI, agentic AI, context, risk - for practitioners.

~25 min

The Copilot platform

Every surface and agent: IDE, cloud agent, CLI, Spaces, Spark, review, security.

~40 min

Across the SDLC

The map to Modules 2-7, an end-to-end example, and the evidence.

~25 min

Learning objectives

approx 2 min

By the end of this module you will be able to:

Reason about modern AI

Explain generative vs agentic AI precisely, and judge where each fits - beyond buzzwords.

Navigate the Copilot platform

Describe Copilot's surfaces and agents (IDE, cloud agent, CLI, Spaces, Spark, review) and when to use each.

Use it responsibly at work

Apply enterprise guardrails: data boundaries, IP, security, human review of agent output.

Connect it to the lifecycle

Locate where Copilot helps in each SDLC phase and which module goes deep on it.

From autocomplete to agents: what changed

approx 4 min

The reason this training is level 300, not the 2022 version:

2021-2023 · Assistant

- Inline autocomplete - one suggestion at a time
- You drive; it completes the current line
- Scope: the file you're in
- Mental model: a very fast typist

2025-2026 · Agent platform

- Takes a goal, plans, edits many files, runs commands
- Works in the IDE, the terminal, and the cloud
- Scope: whole repo, issues, PRs, pipelines
- Mental model: a junior teammate you delegate to

Same underlying idea (LLMs), a fundamentally different way of working. Today reflects the 2026 reality.

The concept stack, precisely

approx 3 min

AI	Software doing tasks that need 'intelligence'.	*e.g.* **Spam filters, recommendations**
Machine Learning	Learns patterns from data, not hand-coded rules.	*e.g.* **Power BI forecasting, defect prediction**
Deep Learning	ML with large neural networks.	*e.g.* **Vision, speech, language**
Generative AI / LLMs	Creates new content; predicts tokens from context.	*e.g.* **Copilot code, DAX, summaries**
Agentic AI	An LLM that plans, uses tools and acts in a loop.	*e.g.* **Copilot cloud agent: issue to PR**

What 'agentic' actually means

approx 4 min

The loop

1. Goal - you state the outcome
2. Plan - break into steps
3. Act - call a tool / edit / run
4. Observe - read results & errors
5. Repeat - until done, then ask you

Tools via MCP (Model Context Protocol)

The agent doesn't just talk - it calls tools. MCP is the open standard that lets it reach your systems:

- Search the repo, open a PR
- Run tests & linters
- Query a database or API
- Read issues / Jira / docs

Autonomy is a spectrum - you choose how much to grant, and approve consequential actions.

LLMs in practice: context is the product

approx 3 min

Prompt + context in, tokens out	The model predicts one token at a time from what it can see. Output quality tracks input quality.
Context window	It only 'knows' what's in the window right now - your open files, chat, attached docs. Big but finite.
Retrieval & grounding	Good tools pull the RIGHT context in (repo search, Spaces, MCP) so answers are grounded, not guessed.
Non-deterministic	Same prompt can differ run to run. Treat output as a draft to verify, never a looked-up fact.

Failure modes - and how practitioners mitigate them

approx 4 min

Hallucination	Invents APIs, functions, facts - confidently.	**Mitigate:** Verify against docs/tests; ground with Spaces/MCP; keep changes small & reviewable.
Insecure code	May reproduce vulnerable patterns.	**Mitigate:** Copilot code review + Autofix + code scanning in CI; never merge unread.
IP / licensing	Could echo public code.	**Mitigate:** Enable the duplication-detection filter; rely on IP indemnity; review attributions.
Over-trust / skill fade	Accepting code you can't explain.	**Mitigate:** Human-in-the-loop; 'explain this'; treat agent PRs like a junior's - review every line.

Responsible use in the enterprise

approx 2 min

Data boundaries

Business/Enterprise: your prompts & code are not used to train models; content exclusion hides sensitive files.

Policy & governance

Admins set org policies, audit logs, and which features/models are allowed.

IP protection

Optional filter blocks suggestions matching public code; IP indemnity on paid plans.

Human accountability

AI drafts; a person reviews and owns what ships. Agent PRs need independent human approval.

PART 2

GitHub Copilot is a platform

Not autocomplete: agent mode, a cloud agent, a CLI, Spaces, Spark, review and security - across every surface.

approx 40 min

One Copilot, many surfaces

approx 3 min

In your editor

VS Code, Visual Studio, JetBrains, Eclipse, Xcode - completions, chat, agent mode.

On github.com

Immersive chat, ask across repos/PRs/issues, deep research, launch agents.

In the terminal

Copilot CLI - an agent that plans, edits files and runs tasks from the shell.

On mobile

GitHub Mobile - chat, watch agent sessions, kick off work on the go.

Desktop app

Agent-native control centre for running and merging agent sessions (preview).

In the cloud

The asynchronous cloud agent works while you do other things.

Three ways Copilot works

approx 3 min

1

Completions

Ghost-text as you type. Tab to accept.

You drive

2

Chat / Edit

Ask, /tests, multi-file edits.

You direct

3

Agents

Give a goal; it plans, acts, iterates.

You delegate

Increasing autonomy - and increasing need for review. Most days you'll blend all three.

Agent mode in the IDE

approx 4 min

In VS Code's Copilot Chat, switch to Agent. Give a goal; it edits across files and runs commands - you review the diff.

You (agent mode)

```
Add server-side rate limiting to
POST /api/login: 5 req/min per IP.
Update the middleware and add tests.
```

Copilot - what it did

```
# edited 3 files
+ middleware/rateLimit.js
~ routes/auth.js   ~ app.js
$ npm test   -> 1 failing -> fixed
```

Why it matters

- **Multi-file:**
- changes middleware, route and app together.
- **Runs & self-heals:**
- executes tests, reads the failure, fixes it.
- **Tools via MCP:**
- can search the repo, hit a DB, open a PR.
- **You stay in control:**
- review the diff, approve terminal commands.

The Copilot cloud agent: issue to pull request

approx 5 min

Assign a GitHub Issue to Copilot. It works asynchronously in its own cloud environment and opens a draft PR you review.

GitHub Issue #482

```
Add rate limiting to login endpoint
Assignees:  @copilot
-> Copilot starts a session,
   pushes commits to a draft PR
```

Draft PR #489

```
branch: copilot/rate-limit-login
3 commits · tests passing
session log: plan -> edit -> test
awaiting your review
```

Guardrails (built in)

- Only pushes to copilot/* branches
- Branch protections & rulesets still apply
- CI on its PRs needs human approval to run
- It cannot approve or merge its own PR
- The person who assigned it can't approve it
- -> an independent human always reviews

Copilot CLI: an agent in your terminal

approx 3 min

```
terminal
$ copilot
> /plan add caching to the reports API
  (reviews plan, then executes steps)

$ copilot -p "bump deps, fix breaks" \
    --allow-all-tools
  (headless / scriptable in CI)

> /delegate   # hand off to cloud agent
```

What it is

- **Terminal-native agent:**
- plans, edits files, runs tests, iterates.
- **Scriptable:**
- headless mode runs inside pipelines & scripts.
- **Plan / Autopilot:**
- choose how much to approve.
- **Model picker + MCP:**
- same tools and models as the IDE.
- **Hand-off:**
- /delegate pushes long jobs to the cloud agent.

Live demo: delegate and review

approx 5 min

Watch for

- Agent mode turning a goal into a multi-file diff
- It running tests and fixing its own failure
- Assigning an issue to @copilot -> a draft PR
- The session log: plan -> act -> observe
- Reviewing critically - we reject something

Flow

- **1.**
- State a goal in agent mode.
- **2.**
- Watch multi-file edit + test run.
- **3.**
- Assign an issue to Copilot.
- **4.**
- Open the draft PR & session log.
- **5.**
- Review - accept good, reject weak.

Copilot code review

approx 3 min

Add Copilot as a PR reviewer (or auto-review via a ruleset). It comments inline with severity and one-click fixes.

PR #489 - Copilot review comment

```
routes/auth.js  line 42
[ HIGH ] SQL injection risk
user input concatenated into query.
- const q = "...WHERE id=" + id;
+ db.query("...WHERE id=?", [id]);
[ Commit suggestion ]
```

Where it fits

- **Fast first pass:**
- catches issues before a human looks.
- **Severity + grouping:**
- High/Medium/Low, less noise.
- **One-click fixes:**
- apply a suggestion in the PR.
- **Not a replacement:**
- a human still approves the merge.

Important: a Copilot review is an assistant pass - it does not satisfy a required-human-reviewer rule. People still own approval.

Copilot Spaces: engineer the context

approx 3 min

A shareable container of the RIGHT context - repos, docs, and instructions - so Copilot answers like a team expert.

```
Space: "Payments Service"
repos:
  acme/payments, acme/payments-docs
files:
  architecture.md, coding-standards.md
instructions:
  Use our conventions. Prefer TypeScript.
  Cite the files you used.
```

Why teams use them

- **Grounded answers:**
- replies use your code & standards, not guesses.
- **Evergreen:**
- GitHub sources auto-update - no stale copy.
- **Onboarding & SMEs:**
- share expertise self-serve, not via Slack pings.
- **Shareable & governed:**
- org-owned with access roles.

Spaces replaced Copilot Enterprise knowledge bases. Available on all plans, including Free.

GitHub Spark: from prompt to running app

approx 3 min

Prompt

```
Build an internal tool to browse and
filter support tickets by status and
owner, with a chart of tickets per week.
```

You get a full-stack app

- UI + data store + auth + AI, hosted
- One-click deploy; iterate by prompt, UI, or code
- Sync to a repo; hand off to the cloud agent

Where it fits

- **Prototyping:**
- turn an idea or a screenshot into a working app fast.
- **Internal tools:**
- the small apps that never get built otherwise.
- **Design-to-code:**
- a real path from mockup to interactive.
- **Included:**
- part of Copilot; each prompt uses credits (preview).

Security: Autofix & secret scanning

approx 3 min

Code scanning alert -> Copilot Autofix

```
CodeQL: SQL injection (High)
- "SELECT * FROM users WHERE id="+id
+ parameterized query with binding
suggested fix on the PR, pre-tested
```

Two AI security helpers

- **Autofix:**
- turns a scanning alert into a suggested fix on the PR.
- **Secret scanning:**
- AI finds generic passwords pattern-matching misses.
- **In CI:**
- fixes and alerts land where you already review.

>3x faster median vulnerability remediation with Autofix (28 min vs ~1.5 hr); up to 7x for XSS, 12x for SQL injection.

Source: GitHub Advanced Security / Code Security. No Copilot subscription required for Autofix on public repos.

Model choice - one Copilot, many models

approx 2 min

OpenAI

GPT-5 series - strong general default

Anthropic

Claude 4 series (Sonnet / Opus) - deep reasoning & agents

Google

Gemini 3 - long context on some surfaces

BYOK / local

Bring your own key or run a local model (Business/Enterprise)

Pick per task with a model picker (or 'Auto'); admins can restrict models. Versions change fast - the point is choice, not a specific model.

Editions, credits & governance

approx 2 min

Plans (individual -> enterprise)

- **Free:**
- limited completions/chat, single model.
- **Pro / Pro+ / Max:**
- individuals; more usage & model access.
- **Business ($19) / Enterprise ($39):**
- orgs - policies, pooled usage, admin controls.
- **Usage-based credits:**
- metered by tokens; completions stay free; budgets per org.

Enterprise governance

- **Policies:**
- enable/disable features & models org-wide.
- **Audit logs:**
- track Copilot usage for compliance.
- **Content exclusion:**
- hide sensitive files/repos.
- **Metrics API:**
- adoption & usage dashboards, per-user credits.
- **Spaces & instructions:**
- org-wide context and conventions.

PART 3

Copilot across the SDLC

The map to the rest of this course, one end-to-end example, and what the evidence says.

approx 25 min

Where Copilot helps - and which module goes deep

approx 5 min

Requirements

Draft issues & specs from notes; break down work.

Module 2

Design

Diagrams from code; prototype apps with Spark.

Module 3

Development

Agent mode, cloud agent, completions, chat.

Module 4

Testing

Generate & maintain tests via chat/agents.

Module 5

Deployment

Explain/fix failing Actions; agentic workflows.

Module 6

Maintenance

Code review, Autofix, log/incident agents.

Module 7

Today = the tool and the concepts. The deep, hands-on techniques for each phase live in Modules 2-7.

One feature, end to end

approx 5 min

How the pieces connect on a single change - each step is a Copilot surface you met in Part 2.

1. Plan

Copilot drafts issue #482 & sub-tasks from a request.

2. Delegate

Assign to @copilot -> draft PR on copilot/* branch.

3. Review

Copilot code review flags a High issue; you fix it.

4. Secure

Autofix resolves a scanning alert on the PR.

5. Ship

A human approves & merges; CI deploys.

A human is in the loop at review and merge - the agent accelerates the work, people own the decisions.

What the evidence says

approx 4 min

55%

faster task completion in a controlled study of 95 developers

+84%

more successful builds in Accenture's enterprise rollout

3x

faster vulnerability fixes with Copilot Autofix

90%

of the Fortune 100 use Copilot (mid-2025)

Read critically: gains are real but vary by task, seniority and codebase - some tasks see little benefit. Measure your own baseline rather than quoting vendor numbers.

Sources: GitHub developer-productivity study; GitHub x Accenture research; GitHub Advanced Security; Microsoft FY25 Q4.

Where it's strong - and where it isn't (yet)

approx 3 min

Strong today

- Writing, refactoring & explaining code
- Test generation and boilerplate
- Well-scoped, delegated changes (cloud agent)
- Security fixes and code review passes
- Grounded Q&A over your code (Spaces)

Still maturing

- Deep design judgement & novel architecture
- Large, cross-cutting changes without guidance
- Ops/observability (extensible, not turnkey)
- Domain nuance without curated context
- Anything unreviewed - accountability stays human

How we'll use Copilot in your labs

approx 2 min

Web / Drupal

Agent mode & completions in VS Code; delegate chores to the cloud agent; review its PRs.

Data & SQL

Chat & CLI for queries and transforms; Spaces to ground answers in your schema.

Power BI

Copilot in Power BI for measures & narratives; chat to explain and document models.

QA

Generate and maintain tests via chat/agents; Copilot code review on PRs.

Discussion & reflection

approx 3 min

Talk to the person next to you - two minutes:

- Which Copilot surface (agent mode, cloud agent, CLI, Spaces) would change your week the most?
- What would need to be true - guardrails, data, process - for your team to trust delegating to an agent?

Trust & review	Data boundaries	Skill impact	Governance	Where to start

We'll take two or three answers before wrapping up.

Key takeaways

- Agentic AI = an LLM that plans, uses tools and acts - not autocomplete.
- Copilot is a platform: IDE agent, cloud agent, CLI, Spaces, Spark, review, security.
- It spans the whole SDLC - each phase goes deep in Modules 2-7.
- Guardrails make delegation safe: independent review, protections, data boundaries.
- AI drafts; you decide. Ground the context, review every change.

What's next

Module 2

Requirements (Planning & Analysis)

then

Lab 1 this afternoon

Questions before the break?

Thank you.

Abhimanyu Singhal

Resources
slides: https://1drv.ms/f/c/e979b4f5ba69eb89/IgApvqNi3RrrTYSY8UqfHHQ_ASRkY1PZ28F4ry0XK0IbtRc?e=1xbDNl
Github: abhimanyusinghal/github_copilot_sdlc
Docs: GitHub Copilot documentation - GitHub Docs
MS Learn:
https://learn.microsoft.com/training/paths/copilot/
https://learn.microsoft.com/training/paths/gh-copilot-2/